

HANDS-ON WORKSHOP · FASTAPI FUNDAMENTALS

Building a Banking API with FastAPI

A step-by-step, file-by-file walkthrough of the banking_api project — for freshers and FastAPI beginners

Routers

Services

Repositories

SQLAlchemy ORM

Pydantic

No prior FastAPI experience needed · Every step maps to a real file you can open



What we are building

Six HTTP endpoints over one SQLite table called accounts

Method	Path	What it does
GET	/	Welcome message + app name
POST	/accounts/	Create account (balance starts at 0)
GET	/accounts/	List every account
GET	/accounts/{account_id}	Fetch one account, 404 if missing
POST	/accounts/{account_id}/deposit	Add money (amount must be > 0)
POST	/accounts/{account_id}/withdraw	Remove money, refuses overdraft

By the end you can...

- Explain the job of every file and folder
- Trace one HTTP request end to end
- Read Depends() chains with confidence
- Predict the status code (200/400/404/422) before you hit Send
- Add a new feature in the right layer



Python you need first (5-minute primer)

If these six ideas are clear, the whole project is readable

Type hints

`account_id: int`
FastAPI reads hints to decide where data comes from and to validate it. Hints are not just documentation here.

Classes & `__init__`

`AccountService` and `AccountRepository` are classes. Their constructor receives the object they depend on.

Decorators

`@router.get("/")` wraps a function and registers it as an endpoint. The function itself stays normal Python.

Generators (yield)

`get_db()` yields a session, pauses, then resumes to run finally: `db.close()` once the request is done.

Packages & relative imports

`from ..database import Base`
`.` = this package `..` = parent package.
Needs `__init__.py` and running as a package.

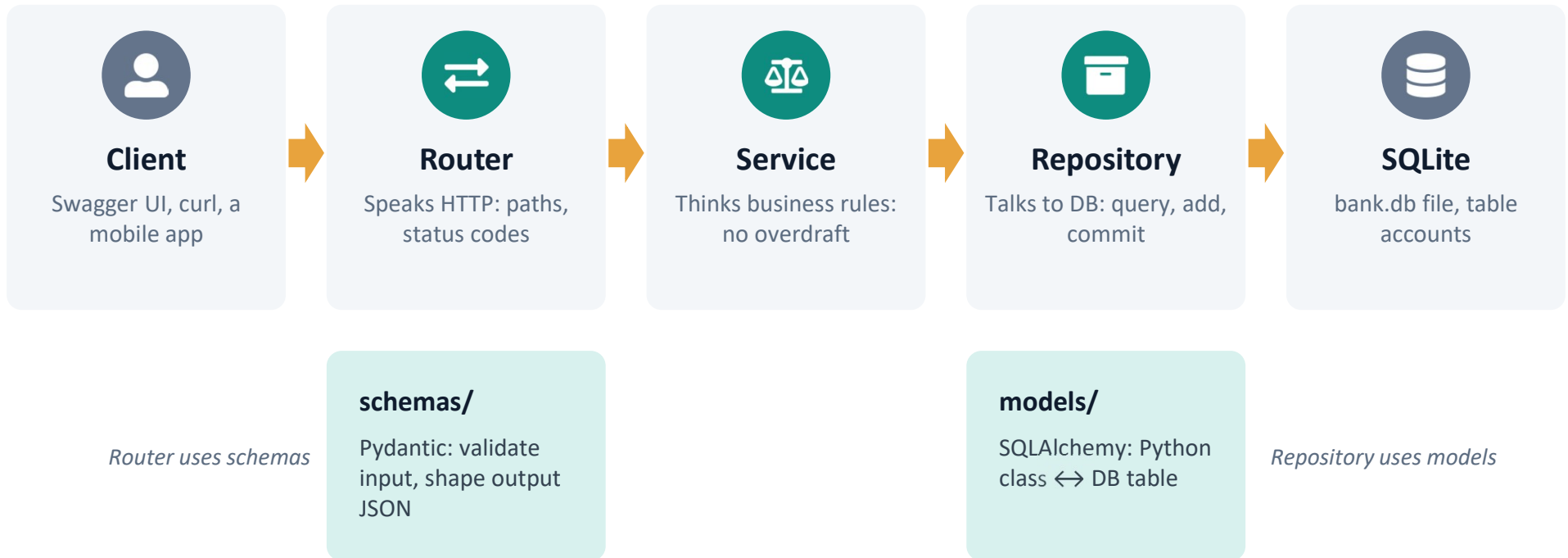
Optional / List

`Optional[Account]` means "an Account or None". `List[AccountOut]` means "a list of AccountOut".



The big picture: one request, four layers

Each layer talks only to its neighbour



Foundation: config.py (settings) + database.py (engine, sessions, get_db) + main.py (wires it all)



Folder structure: one folder, one job

Open the project in your editor and follow along

```
banking_api/
├── .env                # config values
├── requirements.txt    # dependencies
├── bank.db             # SQLite file (created)
├── app/               # the Python package
│   ├── __init__.py    # marks it a package
│   ├── main.py        # creates FastAPI app
│   ├── config.py      # Settings class
│   ├── database.py    # engine, session, Base
│   ├── models/account.py
│   ├── schemas/account_schema.py
│   ├── repositories/account_repository.py
│   ├── services/account_service.py
│   └── routers/account_router.py
```

Folder	Question it answers
routers/	Which URL? Which status code?
services/	Is this allowed? (rules)
repositories/	How do I read/write the DB?
schemas/	What JSON comes in / goes out?
models/	What does the table look like?

Every sub-folder also has an empty `__init__.py` so Python treats it as a package and relative imports work.



An analogy: a bank branch

The code is organised the way a real branch divides work



Counter clerk = Router

Takes your slip, checks it is filled correctly (schema), hands it on, and gives you the receipt or the rejection.



Branch manager = Service

Applies the rules: amount must be positive, no withdrawal beyond balance. Never touches the ledger directly.



Records clerk = Repository

The only person allowed to open the ledger: find account, write entry, save.



Ledger page layout = Model

Fixed columns: id, owner_name, balance. The physical book is the SQLite file.

Why bother splitting?

- Test rules without a web server
- Swap SQLite for PostgreSQL by changing one URL
- Add a CLI or scheduled job that reuses the same Service
- New joiners find code by asking "HTTP, rules, or storage?"



Step 0 · Set up and run

Virtual environment → install → start the server

```
terminal

# from inside banking_api/
python -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt

uvicorn app.main:app --reload
# open http://127.0.0.1:8000/docs
```

uvicorn **app.main:app** **--reload**

app.main = module path (app/main.py), imported as a package

app = the variable named app inside that module

--reload = restart on code change (development only)

Package	Role
fastapi	Web framework: routing, validation, docs
uvicorn	ASGI server that actually runs the app
SQLAlchemy	ORM: Python objects ↔ SQL rows
pydantic	Data validation via type hints
pydantic-settings	Reads env vars and .env into Settings

1

Step 1 · config.py — settings in one place

pydantic-settings turns environment values into a typed object

```
app/config.py

from pydantic_settings import (BaseSettings,
                               SettingsConfigDict)

class Settings(BaseSettings):
    DATABASE_URL: str = "sqlite:///./bank.db"
    APP_NAME: str = "Personal Banking API"
    model_config = SettingsConfigDict(
        env_file=".env", env_ignore_empty=True)

settings = Settings()
print(f"Starting {settings.APP_NAME}")
```

Priority (highest wins): 1 real environment variable → 2 .env file → 3 default in the class

1

Typed fields

Each class attribute is a setting with a type and a default value.

2

env_file=".env"

Values in .env override the defaults. Our .env sets APP_NAME to "... (Local Server)".

3

env_ignore_empty=True

APP_NAME="" in the environment is ignored, so the .env / default value is used.

4

Module-level singleton

settings = Settings() runs once on first import; every file shares that object.

2

Step 2 · database.py — engine, sessions, Base

The plumbing every other layer relies on

```
app/database.py

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from .config import settings

connect_args = ({ "check_same_thread": False }
                 if settings.DATABASE_URL.startswith("sqlite") else {})
engine = create_engine(settings.DATABASE_URL,
                       connect_args=connect_args)
SessionLocal = sessionmaker(autocommit=False,
                             autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    db = SessionLocal()      # 1. open
    try:
        yield db             # 2. lend to the request
    finally:
        db.close()          # 3. always close
```

1

engine

Knows HOW to connect (URL, driver, pool).
Created once for the whole app.

2

check_same_thread=False

Plain def endpoints run in a thread pool; SQLite would otherwise refuse a connection used from another thread.

3

SessionLocal

A factory. Each call gives a new Session = one unit of work. autocommit=False → you must call commit().

4

Base

Parent class for all ORM models; it collects their table definitions in Base.metadata.

5

get_db()

A yield dependency: one session per request, closed in finally even if an error occurs.

3

Step 3 · models/account.py — the table as a class

SQLAlchemy ORM maps Python attributes to columns

```
app/models/account.py

from sqlalchemy import Column, Integer, String, Float
from ..database import Base

class Account(Base):
    __tablename__ = "accounts"
    id = Column(Integer, primary_key=True, index=True)
    owner_name = Column(String, index=True, nullable=False)
    balance = Column(Float, default=0.0, nullable=False)
```

Resulting SQLite table: accounts

column	SQL type	rule
id	INTEGER	PK, indexed
owner_name	VARCHAR	NOT NULL, indexed
balance	FLOAT	NOT NULL

class Account



one table

__tablename__



table name in SQL

one object



one row

primary_key=True



SQLite auto-assigns id on insert

4

Step 4 · schemas — the API's contract

Pydantic models describe the JSON, not the table

```
app/schemas/account_schema.py

from pydantic import BaseModel, ConfigDict

# request body
class AccountCreate(BaseModel):
    owner_name: str

# response body
class AccountOut(BaseModel):
    id: int
    owner_name: str
    balance: float
    model_config = ConfigDict(
        from_attributes=True)

# in account_router.py
class AmountRequest(BaseModel):
    amount: float
```

	Model (SQLAlchemy)	Schema (Pydantic)
Describes	DB table	JSON in / out
Base class	Base	BaseModel
Used by	Repository	Router
Example	Account	AccountCreate, AccountOut

Why two input/output schemas?

AccountCreate has no id or balance, so a client cannot choose them. Tested: sending {"owner_name": "B", "balance": 9999} creates B with balance 0.0; extra keys are ignored. from_attributes=True lets AccountOut read account.id from an ORM object instead of a dict.

5

Step 5 · repositories — data access

The only layer that touches the database

```
app/repositories/account_repository.py

class AccountRepository:
    def __init__(self, db: Session = Depends(get_db)):
        self.db = db

    def get_by_id(self, account_id: int) -> Optional[Account]:
        return (self.db.query(Account)
                .filter(Account.id == account_id).first())

    def get_all(self) -> List[Account]:
        return self.db.query(Account).all()

    def create(self, account: Account) -> Account:
        self.db.add(account)          # stage the new object
        self.db.commit()              # write the transaction
        self.db.refresh(account)      # reload: now has its id
        return account

    def update(self, account: Account) -> Account:
        self.db.commit()              # object is already tracked
        self.db.refresh(account)
        return account
```

1

Session injected

Depends(get_db) as the default value:
FastAPI supplies a fresh session for each request.

2

first() vs all()

first() gives one object or None; all() gives a list (maybe empty).

3

add → commit → refresh

refresh re-reads the row so the DB-generated id and stored values are on the object.

4

update() has no add()

The account came from this same session, so the session already tracks its changes.

6

Step 6 · services — the business rules

Pure Python decisions; no HTTP knowledge at all

```
app/services/account_service.py

class AccountService:
    def __init__(self,
                  repo: AccountRepository = Depends()):
        self.repo = repo

    def withdraw(self, account_id: int, amount: float):
        if amount <= 0:
            raise ValueError(
                "Withdraw amount must be positive")
        account = self.repo.get_by_id(account_id)
        if not account:
            return None
        if account.balance - amount < 0:
            raise ValueError("Insufficient funds")
        account.balance -= amount
        return self.repo.update(account)
```

Service signals with Python: raise ValueError or return None. The router translates those into HTTP.

Checks run in this order

amount <= 0 ?

→ ValueError

400

account missing ?

→ return None

404

balance - amount < 0 ?

→ ValueError

400

otherwise

→ subtract + commit

200

7

Step 7 · routers — HTTP in, HTTP out

Decorators map URL + method to a Python function

```
app/routers/account_router.py (reformatted)

router = APIRouter(prefix="/accounts", tags=["Accounts"])

@router.get("/{account_id}", response_model=AccountOut)
def get_account(account_id: int,
                 service: AccountService = Depends()):
    account = service.get_account(account_id)
    if not account:
        raise HTTPException(status_code=404,
                             detail="Account not found")

    return account

@router.post("/{account_id}/withdraw", response_model=AccountOut)
def withdraw_money(account_id: int, request: AmountRequest,
                   service: AccountService = Depends()):
    try:
        updated = service.withdraw(account_id, request.amount)
        if not updated:
            raise HTTPException(404, "Account not found")
        return updated
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
```

1

prefix + tags

All paths start with /accounts; tags group them in Swagger.

2

Where does data come from?

{account_id} in path → path param. A
BaseModel param → JSON body.

3

Depends() with no argument

FastAPI uses the type hint, AccountService, as the dependency to build.

4

response_model

Validates and filters the return value into AccountOut JSON.

5

try / except ValueError

HTTPException is not a ValueError, so the 404 passes straight through.

8

Step 8 · main.py — wiring it together

The entry point uvicorn loads

```
app/main.py (message shortened)

from fastapi import FastAPI
from .routers.account_router import (
    router as account_router)
from .database import Base, engine
from .config import settings

app = FastAPI(title=settings.APP_NAME)

@app.on_event("startup")
def on_startup():
    # Auto-create tables (for demo convenience)
    Base.metadata.create_all(bind=engine)

@app.get("/")
def root():
    return {"app": settings.APP_NAME,
            "message": "Welcome to the ..."}

app.include_router(account_router)
```

1

FastAPI(title=...)

The title appears in /docs and in openapi.json.

2

Hidden import chain

Importing the router imports service → repository → models.account, which registers Account on Base.metadata.

3

create_all at startup

Creates tables that do not exist yet. It never alters an existing table (that is what migrations are for).

4

include_router

Mounts every /accounts route onto the app.

5

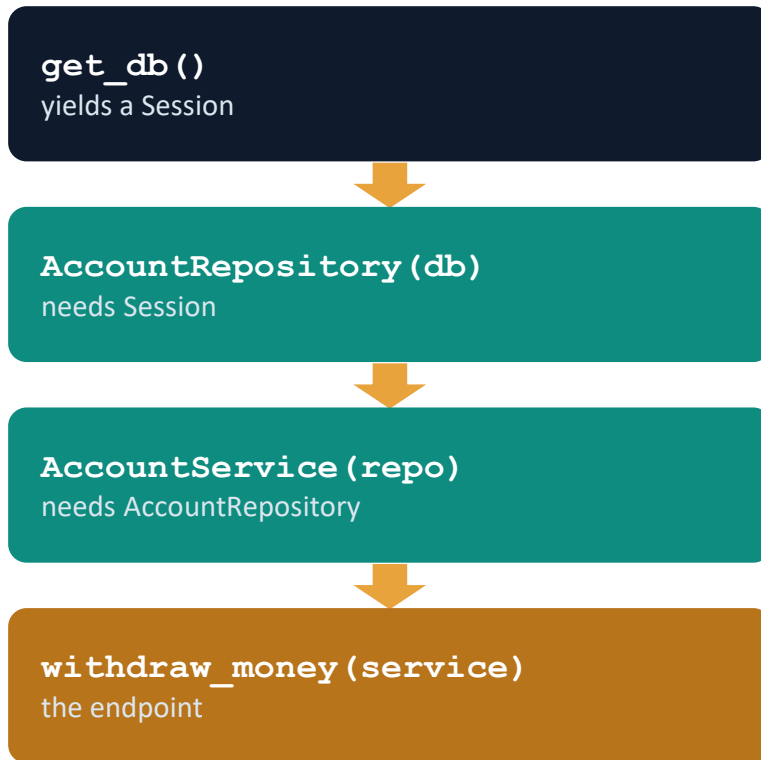
on_event is deprecated

Modern FastAPI prefers a lifespan function; works here but shows a warning.



Deep dive · Dependency Injection with Depends()

FastAPI builds the object graph for you, per request



- 1 Resolved bottom-up**
FastAPI reads the signatures, sees each needs something else, and builds `get_db` → `repo` → `service` before calling the endpoint.
- 2 Cached per request**
Within one request a dependency is created once. We measured `get_db` called exactly 1 time per request.
- 3 Cleanup after**
After the endpoint finishes, FastAPI resumes `get_db` past its yield and closes the session.
- 4 Swap in tests**
`app.dependency_overrides[get_db] = fake_db` lets tests use a test database with no code change.



Trace one request end to end

POST /accounts/1/withdraw body {"amount": 200}

1

Uvicorn

receives the HTTP request and hands it to the FastAPI app

2

Routing

matches POST + /accounts/{account_id}/withdraw → withdraw_money

3

Dependencies

get_db() opens Session → AccountRepository → AccountService

4

Validation

"1" → int 1; JSON → AmountRequest(amount=200.0).
Bad input → 422, endpoint body never runs

5

Service

amount > 0 ✓, repo.get_by_id(1) ✓, balance ≥ 200 ✓,
balance -= 200

6

Repository

commit() → SQL UPDATE accounts ... ; refresh() re-reads the row

7

Response

Account object → response_model AccountOut → JSON, status 200

8

Cleanup

get_db resumes after yield: finally → db.close()



Predict the status code

All rows below were run against this exact project

Code	Who produces it	Example request	Response detail
200	Your endpoint returns normally	POST /accounts/1/deposit {amount:100}	Updated AccountOut
400	Router: ValueError → HTTPException	POST /accounts/1/withdraw {amount:1000}	"Insufficient funds"
404	Router: service returned None	GET /accounts/999	"Account not found"
422	FastAPI validation, before the endpoint runs	GET /accounts/abc	int_parsing error on path.account_id
422	FastAPI validation	POST /accounts/1/withdraw?amount=5	body: Field required
405	FastAPI: path exists, method doesn't	PUT /accounts/1	Method Not Allowed
307	Redirect to trailing-slash path	GET /accounts	Location: /accounts/



Hands-on Lab 1 · Swagger UI (/docs)

Predict first, then click Try it out → Execute

1. Create two accounts: "Asha" and "Ravi". Note the ids returned.
2. Deposit 1000 into Asha. Then withdraw 1500. Predict code and message.
3. Withdraw exactly 1000. Is a zero balance allowed?
4. Call GET /accounts/abc. Which part of the error JSON tells you where it failed?
5. Deposit -50 into account 999. 400 or 404? Why?
6. Create an account with body {"owner_name": "X", "balance": 5000}. What balance comes back?

Answers (reveal later)

2 → 400 "Insufficient funds"

3 → 200, balance 0.0 (rule is < 0)

4 → 422, loc: ["path", "account_id"]

5 → 400: amount checked before lookup

6 → balance 0.0: extra field ignored

L2

Hands-on Lab 2 · the same calls from the terminal

Swagger is just a UI over plain HTTP

```
terminal

# create
curl -X POST http://127.0.0.1:8000/accounts/ \
  -H "Content-Type: application/json" \
  -d '{"owner_name": "Asha"}'
# -> {"id":1,"owner_name":"Asha","balance":0.0}

# deposit
curl -X POST http://127.0.0.1:8000/accounts/1/deposit \
  -H "Content-Type: application/json" -d '{"amount": 500}'
# -> {"id":1,"owner_name":"Asha","balance":500.0}

# list (note the trailing slash)
curl http://127.0.0.1:8000/accounts/

# inspect the actual database file
sqlite3 bank.db "SELECT * FROM accounts;"
```

Try without the slash

`curl -i http://127.0.0.1:8000/accounts`
You get 307 and a Location header. Add
-L to follow it.

Look at the contract

Open `/openapi.json`. Every schema and
path you saw in code is described there;
Swagger UI is generated from it.



Gotchas we actually hit in this code

Each one was reproduced while preparing this session



Float money

Deposit 0.1 then 0.2 → balance 0.30000000000000004. Use Decimal in Python and Numeric in the DB for money.



python app/main.py fails

ImportError: attempted relative import with no known parent package. Run `uvicorn app.main:app` from `banking_api/`.



Relative SQLite path

`sqlite:///./bank.db` is relative to where you START the server. Launch elsewhere → a new, empty `bank.db`.



Trailing slash

Routes are `/accounts/`. GET `/accounts` answers 307 redirect; some clients don't follow redirects on POST.



Empty names accepted

`owner_name: str` allows `""`. Add `Field(min_length=1)` to reject it with 422.



Deprecated startup hook

`@app.on_event("startup")` still works but emits a `DeprecationWarning`; use `lifespan` instead.

L3

Hands-on Lab 3 · add a transfer feature

Decide which layer each piece belongs to, then write it

```
suggested solution – service layer

# services/account_service.py
def transfer(self, from_id: int, to_id: int, amount: float):
    if amount <= 0:
        raise ValueError("Transfer amount must be positive")
    if from_id == to_id:
        raise ValueError("Cannot transfer to same account")
    src = self.repo.get_by_id(from_id)
    dst = self.repo.get_by_id(to_id)
    if not src or not dst:
        return None
    if src.balance - amount < 0:
        raise ValueError("Insufficient funds")
    src.balance -= amount
    dst.balance += amount
    self.repo.update(src)    # ONE commit saves both
    return src
```

Schema

TransferRequest(to_account_id: int, amount: float)

Service

Rules above: positive, not same, both exist, enough funds

Repository

No new method needed; one commit covers both rows

Router

POST /accounts/{account_id}/transfer → map None to 404, ValueError to 400



From classroom to production: what's missing

A good design to learn on; not yet safe for real money



Money types

Decimal / Numeric(12,2) instead of Float



Auth

OAuth2 / JWT so only the owner sees an account



Tests

pytest + TestClient + dependency_overrides



Logging & config

logging instead of print; secrets from env only



Migrations

Alembic instead of create_all for schema changes



Concurrency

Two withdrawals at once can both pass the check; use row locks or atomic UPDATE



Validation

Field(gt=0) on amount, min_length on name



Domain errors

Custom exceptions + exception handlers instead of ValueError

Recap: Step 0 to Step 8

0 **Setup** venv, pip, uvicorn app.main:app

1 **config.py** typed settings from env/.env

2 **database.py** engine, SessionLocal, Base, get_db

3 **models** Account class $\leftrightarrow$ accounts table

4 **schemas** AccountCreate in, AccountOut out

5 **repositories** query / add / commit / refresh

6 **services** rules; raise ValueError or return None

7 **routers** paths, validation, HTTP errors

8 **main.py** app, startup tables, include_router

*Router talks HTTP · Service thinks rules ·
Repository talks SQL*